# 09 | 工程初始化（下）：前端工程与一键启动

Robert · Claude Code 企业级全链路开发实战




你好，我是 Robert。

上一讲我们搭好了后端骨架——Maven 多模块结构、公共基础设施、健康检查接口，`java -jar` 能跑，访问 `/api/v1/health` 返回 200。

这一讲把剩下的补齐：前端 Vue 工程、前后端联通、启动脚本。做完之后，你执行一个 `./start.sh`，后端前端同时起来，浏览器打开就能看到 Hify 的页面。

从零到能跑的完整闭环，这一讲交付完成。

## 前端 Vue 工程搭建

和上一讲后端一样，前端初始化也要分步做。我分成三步：项目骨架、axios 统一请求层、路由和页面空壳。上一讲建立的任务拆解方法论直接复用，不再解释为什么要拆——按依赖关系排序，每步验证后进入下一步。

### 第一步：项目骨架

给 Claude Code 的指令思路：

> 初始化 Hify 前端项目 hify-web。Vue 3 + TypeScript + Vite + Element Plus。目录结构按 CLAUDE.md 中定义的前端结构来。Vite 开发服务器配置代理：/api 请求转发到 localhost:8080。

输出是：

构建通过。创建的文件：

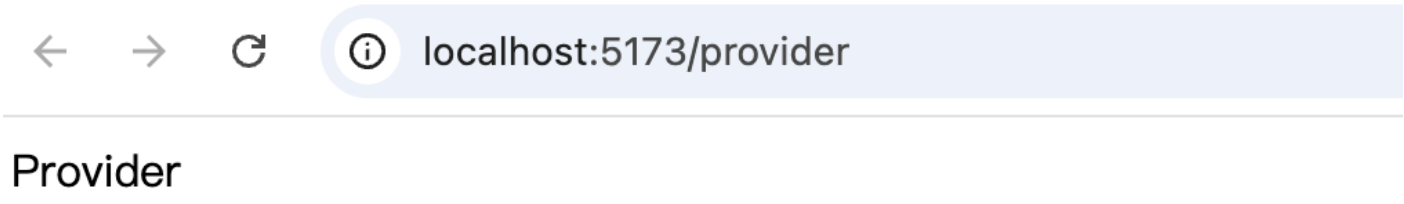
```
hify-web/
├── package.json
├── vite.config.ts          # /api → localhost:8080 代理
├── tsconfig.json
├── tsconfig.node.json
├── index.html
└── src/
    ├── main.ts             # 注册 ElementPlus、Router、Icons
    ├── App.vue
    ├── router/index.ts     # /provider /agent /chat 三条路由
    ├── utils/request.ts    # axios 封装，统一处理 Result.code 错误
    ├── types/index.ts      # PageQuery 等公共类型
    └── views/
        ├── provider/ProviderList.vue
        ├── agent/AgentList.vue
        └── chat/ChatView.vue
```

启动开发服务器：

```
cd hify-web && npm run dev
```

访问 `http://localhost:5173`，`/api` 请求自动转发到后端 8080。

这一步比较标准化，Claude Code 一般不会出大问题。拿到输出后检查两件事：目录结构和 CLAUDE.md 里定义的是否一致；Vite 代理配置是否正确，开发阶段前端跑在 5173 端口，通过 Vite 代理转发 `/api` 请求到后端 8080 端口，解决跨域问题。




是不是很快。说实话，我一点不懂前端。但是通过上面的指令就有一个前端服务了。这里需要注意的是：**提示词要完整且清晰**。你可以参考一下上面的提示词。

### 第二步：axios 统一请求层

这一步值得多讲一下，因为它是前端和后端规范体系的接合点。

后端定了统一响应格式 `Result<T>`，每个接口都返回 `{ code: 200, message: "success", data: {...} }`。前端的 axios 封装要和这套格式对接，让业务代码不需要每次都手动处理 code 判断和 data 解包。

给 Claude Code 的指令思路：

> 在 hify-web/src/utils/ 下创建 request.ts，封装 axios 实例。baseURL 设为 /api。响应拦截器里判断 code：200 直接返回 data 字段（自动解包），非 200 用 Element Plus 的 ElMessage.error 提示 message，然后 reject。导出 get、post、put、del 四个方法。

为什么要在拦截器里自动解包 data？因为如果不这么做，后面每个页面调接口都要写：

复制代码

```
const result = await providerApi.getList()
const list = result.data  // 每次都要 .data
```

封装之后直接就是：

```
const list = await providerApi.getList()  // 直接拿到 data
```

看起来是个小事，但几十个接口累积下来，代码干净很多。来看生成的 request.ts 的代码：

复制代码

```
import axios from 'axios'
import { ElMessage } from 'element-plus'

const instance = axios.create({
  baseURL: '/api',
  timeout: 60000,
})

instance.interceptors.response.use(
  (response) => {
    const { code, message, data } = response.data
    if (code !== 200) {
      ElMessage.error(message || '请求失败')
      return Promise.reject(new Error(message))
    }
    return data
  },
  (error) => {
    ElMessage.error(error.message || '网络异常')
    return Promise.reject(error)
  }
)

export const get = <T>(url: string, params?: object): Promise<T> =>
  instance.get(url, { params })

export const post = <T>(url: string, data?: object): Promise<T> =>
  instance.post(url, data)

export const put = <T>(url: string, data?: object): Promise<T> =>
  instance.put(url, data)

export const del = <T>(url: string): Promise<T> =>
  instance.delete(url)
```

然后让 Claude Code 基于这个封装写一个示例 API 文件：

> 在 hify-web/src/api/ 下创建 health.ts，用封装好的 request 调用 GET /api/v1/health。导出 getHealth 方法。

文件代码是：

```
import { get } from '@/utils/request'

export const getHealth = () => get<string>('/v1/health')
```

这个文件只有几行代码，但它验证了整条链路——axios 封装、代理配置、接口路径规范——是否都通。

回过头看，这就是为什么 06 讲要把接口规范写进 CLAUDE.md，因为不只后端在用，前端 axios 封装也在对着同一份规范做。后端的 `Result<T>` 格式、接口路径规则、错误码定义，前端全部照着来。一份规范，两端对齐。

### 第三步：路由和页面空壳

给 Claude Code 的指令是：

> 在 hify-web 中配置 Vue Router，创建以下路由和对应的空壳页面组件：模型管理、Agent 管理、对话。每个空壳页面只显示页面名称，比如 ProviderList.vue 里就一行"模型提供商管理"。再创建一个 App.vue 布局：左侧 Element Plus 菜单栏（三个菜单项对应三个路由），右侧内容区用 router-view。

这一步生成的是一个有完整导航结构的空壳应用——左边菜单、右边内容，点菜单能切换页面，每个页面都是占位文字。后面往里填内容就行。



## 前后端联通

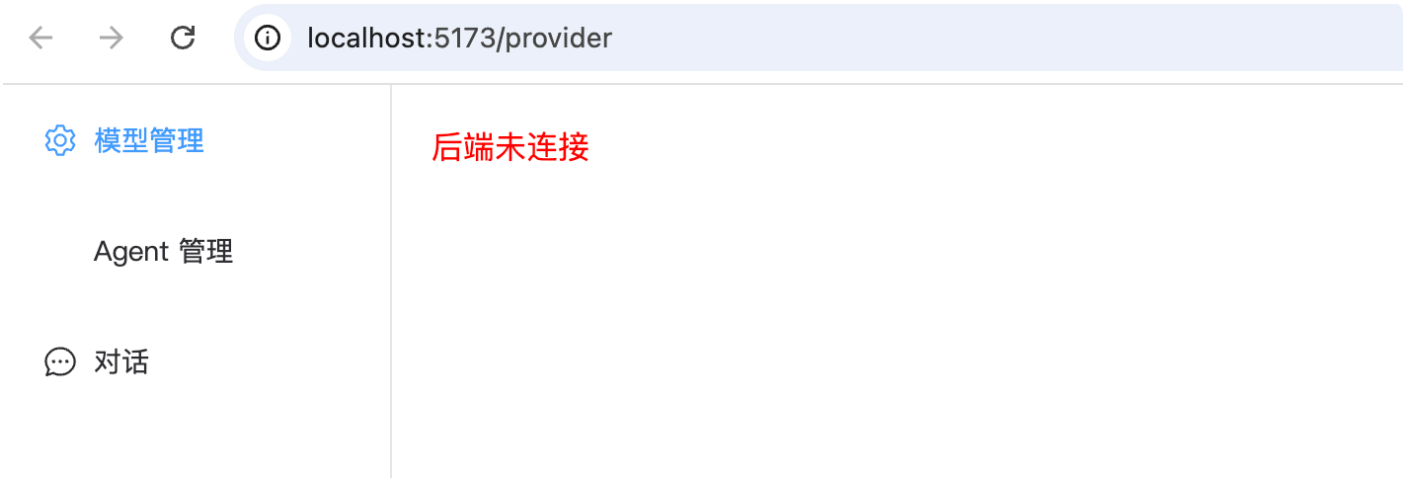
前端搭好了，后端上一讲已经能跑了。现在把它们连起来。

这一步验证的是完整链路：Vite 代理配置对不对、axios 封装能不能用、前后端的接口格式能不能对上。

把前端的 ProviderList.vue 改一下，让它调用健康检查接口并显示结果：

> 修改 ProviderList.vue，在页面加载时调用 getHealth()，把返回结果显示在页面上。如果调用成功显示绿色的"后端已连接：Hify is running"，失败显示红色的"后端未连接"。

启动后端、启动前端，打开浏览器 `http://localhost:5173`，你应该能看到左侧菜单、右侧显示绿色的“后端已连接”。但是，却是：




看了一眼，发现 hify server 被我关了，启动后：




看到了，说明前后端联通成功——axios 封装正常、Vite 代理正常、后端接口正常、Result 格式前端能正确解析。如果显示红色，就把浏览器控制台的错误信息贴给 Claude Code 让它排查。




开发阶段的跨域解决方案，不需要 Nginx
生产环境由 Nginx 反向代理替代 Vite proxy

极客时间

## 启动脚本

到这一步，前后端都能跑了，但每次启动要开两个终端、输两条命令。写个脚本让它一键搞定。

还记得 01 讲的三层分工吗？启动脚本、Makefile 这些就是典型的第三层——AI 全权处理，你验收结果就行。脚本逻辑不复杂，但手写容易漏细节（进程管理、日志输出、端口检查），交

给 Claude Code 特别合适。不需要逐行 review shell 语法，跑一下 `start.sh` 能起来就过了。

**start.sh**

> 写一个 start.sh 脚本，放在项目根目录。功能：检查 MySQL 和 Redis 是否可用，构建后端并后台启动，轮询等待后端健康检查通过，启动前端开发服务器。加上错误处理：任何一步失败就停止并提示。

**stop.sh**

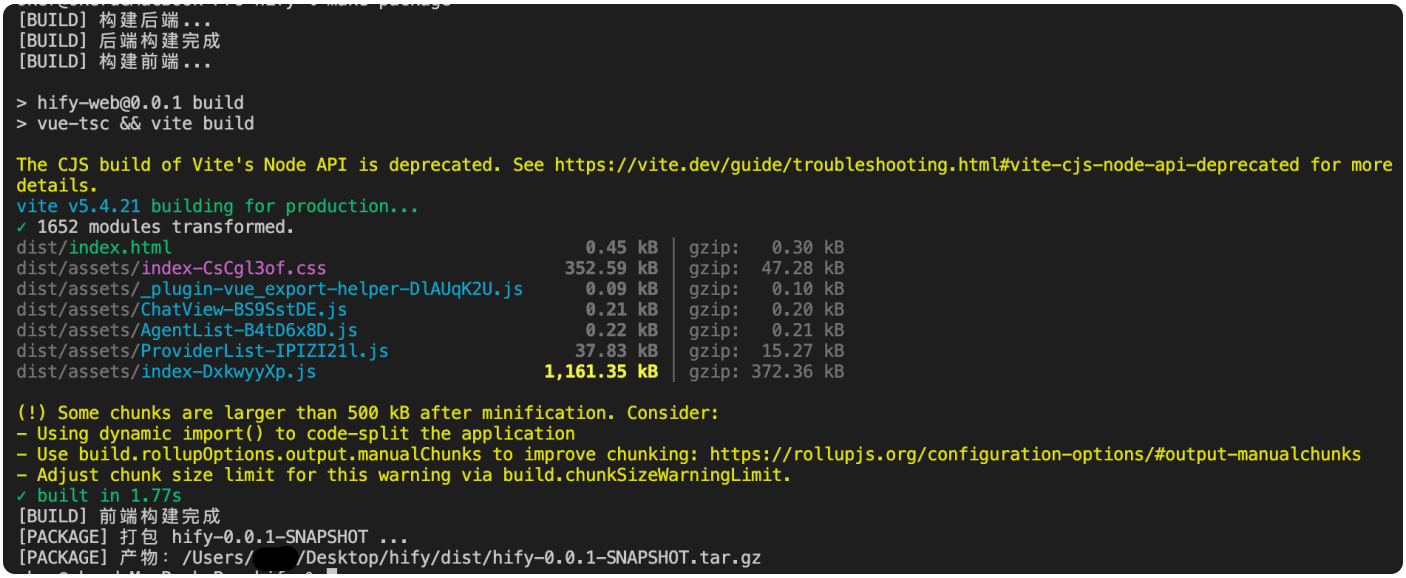
> 写一个 stop.sh 脚本，优雅停止后端和前端进程。按 PID 文件找进程，先 SIGTERM 再等待，超时 SIGKILL。

**Makefile**

> 写一个 Makefile，包含以下 target：make start（启动）、make stop（停止）、make restart（重启）、make build（构建后端 + 前端）、make clean（清理构建产物）、make package（打包成可分发的 tar.gz）。

这里不展示输入和输出的细节了，实操课中会演示，直接展示结果。

这 3 个文件加起来可能就一两百行脚本，但它们让整个项目的使用体验提升了一个档次。拉下代码，`make start` 就跑起来了，`make package` 就能打包分发。




## 验收时刻

所有东西都就绪了。最终验收：

```
sh start.sh
```

你应该看到这样的输出：

```
-e [INFO]  检查 MySQL localhost:3306 ...
-e [ERROR] MySQL 不可达（localhost:3306），请先启动 MySQL
```

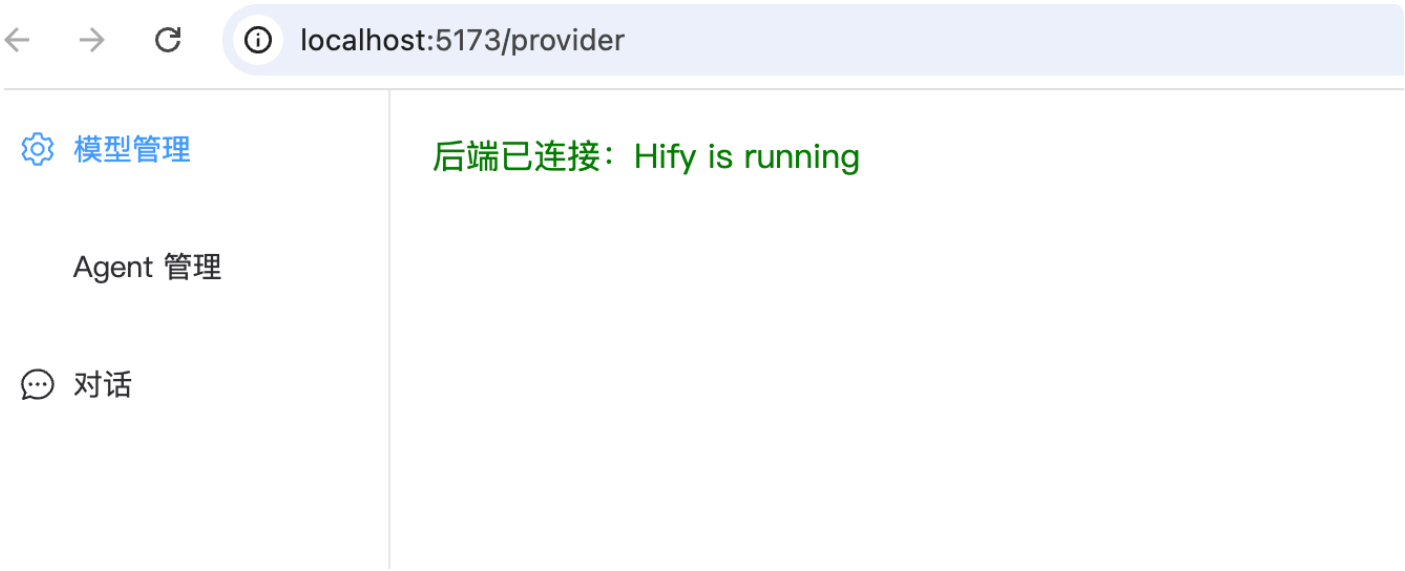
这里是因为 MySQL 没配置。调整配置后，打开浏览器访问 `http://localhost:5173`：

左侧看到三个菜单：模型管理、Agent 管理、对话

点“模型管理”，右侧显示绿色的“后端已连接：Hify is running”

点其他菜单，显示对应的占位文字

看到这些，工程初始化就完成了。你手上有了一个前后端都能跑的空项目，结构清晰、规范就绪、一键启动。后面的工作就是往这个骨架里填功能。




## 总结

我们用两节课完成了 Hify 的工程初始化——后端能跑、前端能开、一键启动、前后端联通。最终，你手上有了一个完整的“空项目。

最后我们回顾一下这两讲的方法论：

**任务拆解。**工程初始化体量大，不能一条指令搞定。按依赖关系拆成有序的步骤，每步验证后再进入下一步。判断标准：生成量超出一次 review 范围、步骤间有依赖，满足其一就拆。

**基础设施先行。**先搭 Maven 骨架、再搭公共模块、再搭业务空壳、再搭前端、最后写脚本。顺序不能乱，因为后面的每一步都依赖前面的结果。

**每步验收。**不是搭完全部才验收，而是每一步做完就验证。每一个验收点给你信心：到这里是对的。

不过，工程初始化完成不代表可以直接写业务了。现在的空项目还缺一层东西——基础组件。数据库怎么做分页、接口入参怎么校验、调外部 API 的客户端怎么封装、线程池怎么配、熔断怎么搞，这些是所有业务模块都会用到的底层能力。

不过下一讲我们要换一种协作模式，不是你告诉 Claude Code 做什么，而是先让它告诉你该做什么。

## 思考题

这两讲工程初始化的过程中，Claude Code 做了大量“模板性”的工作，包括项目结构、配置文件、公共组件、脚本。回想一下你自己平时做项目，有哪些类似的不难但繁琐的工作？列出 3 个你觉得最适合交给 AI 做的，以及你会怎么给它描述需求。

期待你的分享。如果今天的课程让你有所收获，也欢迎转发给有需要的朋友，邀请他来一起学习，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。 页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

## 全部留言 (9)

最新　精选



**～风铃～**
2026-04-20 来自江西

老师，我有一个问题，现在是AI时代，我看有一些AI review代码的。为啥您这里还是选择人工review 代码？ 目前AI review代码不可靠么？ 还是现在还是需要人来管代码？
能不能做到人不管代码，不用看代码，只管功能是否是实现？ review代码也用AI去review

> 作者回复: 要回答这个问题，需要把27讲的内容提前讲一下。我贴一下27讲的一些内容。期待后面的交流～。
>
> Anthropic 公开说过：Claude Code 自身的代码，全部由 AI 编写，工程师只负责 review。在我看来，这句话有一个没说出来的前提：他们敢这么做，不是因为 AI 不会出错，而是因为有一套完整的质量保障体系。
> 在说这套体系之前，我想先说清楚一个判断。
> 现在很多新闻告诉我们：写好 SDD，直接扔给 AI，它跑一个晚上就把整个项目搞定了。我的个人体感是：做不到，至少现在做不到。
> 不是 AI 能力不够，而是这句话偷换了一个概念：保证代码质量的不是 AI，是体系。AI 只是体系的执

行者。

这两件事的区别很大。

● "AI 帮你保证质量"的意思是：你把代码交给 AI，它会自动发现所有问题，自动写出正确的测试，自动保证系统稳定。这是不现实的。AI 写代码时专注于功能实现，它不会主动质疑自己的安全模型，不会想到"这个接口的权限校验是不是有漏洞"，不会考虑并发场景下的数据一致性。

● "体系保证质量，AI 协助执行"的意思是：你先建立质量保障体系，识别核心链路、划定测试边界、制定 review 策略，然后让 AI 在这个体系里高效执行。体系是你建的，AI 是执行者。

一个项目的质量由四个层次组成。

1. 核心链路识别：哪些是系统的核心路径，哪些是风险集中的地方。这个判断必须由人来做，AI 辅助分析。做错了，后面所有测试都是在错误的地方使劲。

2. 单元测试：覆盖纯逻辑，不依赖外部系统。这部分独立性强、模式固定，AI 能规划得很好，可以大比例交给 AI 执行。但要注意：AI 偏逻辑完备，测试用例容易过多，你要把握边界，砍掉不必要的。

3. 集成测试：功能维度的端到端验证，从 Controller 到数据库的完整链路。mock 策略要你来定，执行让 AI 来做，一个场景跑通再写下一个。

4. 混沌测试：故障注入验证韧性，是质量体系的天花板。单测和集成测试验证"正常情况下对不对"，混沌测试验证"出问题时垮不垮"，节点宕机、网络分区、磁盘打满、慢消费者积压。这一层在项目维护阶段才有意义，但要从架构阶段就知道它在哪里。

所以，正确的工作方式不是"告诉 AI：给我写测试用例，保证代码不出问题"然后等它跑完。而是你来拆解项目，划定边界，AI 在你划定的边界里执行。

这一讲围绕这个思路展开。文章分两部分：方法论说清楚怎么想，实操给你能直接用的提示词和流程。








**黑鹰**

2026-04-15 来自德国

请教：将骨架初始化整个流程最后封装成一个Skill，后续再做类似的springboot+vue+mysql这套技术栈的项目，由skill一键初始化，不知是否可行?

作者回复: 是可以的。其实你看，这就升华为了很多开发框架、或通用skill仓库，或skill模板等等。

这门课我希望达到的其实就是你说的这句话效果。就是我们从0开始思考，然后慢慢觉得全部放claude.md太重了，该抽象剥离了，然后此时发现skill很好用，就自然而然用上了。

现在直接讲skill的课很多。我的角度希望从最简单的开始，让大家自然而然去思考我应该用什么，而不是有什么我要用什么。慢慢内化的过程。







**kelly0524**
2026-07-03 来自广东

目前只会java，不会前端知识，所以看到这一章节有个疑问，我是不是得把这块的前端知识去学习一遍才行？不然感觉指挥不了AI去写前端代码

作者回复: 我个人来说：我是后端，我自己对前端不熟，我也不打算去做前端知识。
因为前端的效果直观，对我来说，只要是我要的效果就结束了。而且我觉得前端的bug不会影响稳定性。
也就是核心，前端我全部让ai搞定，我也不准备学。
课程里面的前端提示词可以让ai生成的。我觉得前端是ai做的最好的一个方向。



1



**rogue宇峰**
2026-06-26 来自上海

我还是喜欢ai生成原型图，直接复刻原型图

作者回复: 我也经常这么搞，直接生成HTML 页面。比如：
https://mq9.robustmq.com/
https://robustmq.com/
都是AI 生成的






**星辞Sting**
2026-05-02 来自浙江

常做的：
1. 单元测试
2. 代码commit总结
3. 改动提测时候，撰写修改范围和影响接口功能额



2



**柯察金**
2026-09-01 来自比利时

封装 axios 实例 ， 这些提示，不会前端不太知道咋指挥  ai 了






**Geek_0dc445**

2026-07-17 来自广东

有一个问题，当前测试工程前端和后端都在一个共同的目录中，所以规则管理是一体的，但是很多企业内部的系统前端和后端都是分开的，所以没法共享claude.md，针对这种情况有什么好的建议吗？否则后端的规范改了，但是前端不知道。

共 1 条评论 >



**江流**

2026-04-24 来自四川

我用CC尝试着做了两个新项目，每次都是跑偏，而且修改bug效率不高，现在发现问题就是老师说的：任务粒度拆分的不够细。



**毁灭吧**

2026-04-13 来自北京

已看完，不够看啊老师，我来催更了